



TECNOLÓGICO
NACIONAL DE MÉXICO®



INGENIERIA EN TECNOLOGIAS DE LA INFORMACIÓN Y COMUNICACIONES

PROGRAMACIÓN WEB AVANZADA

ACTIVIDAD 4:

PERSISTENCIA DE DATOS CON
MONGODB/MONGOOSE O
POSTGRESQL/SEQUELIZE

PROFESORA:

ALMA ROSA SANCHEZ AGUIAR

ALUMNA:

YOSELIN DE JESUS SOLIS CERVANTES

NÚMERO DE CONTROL:

221140013

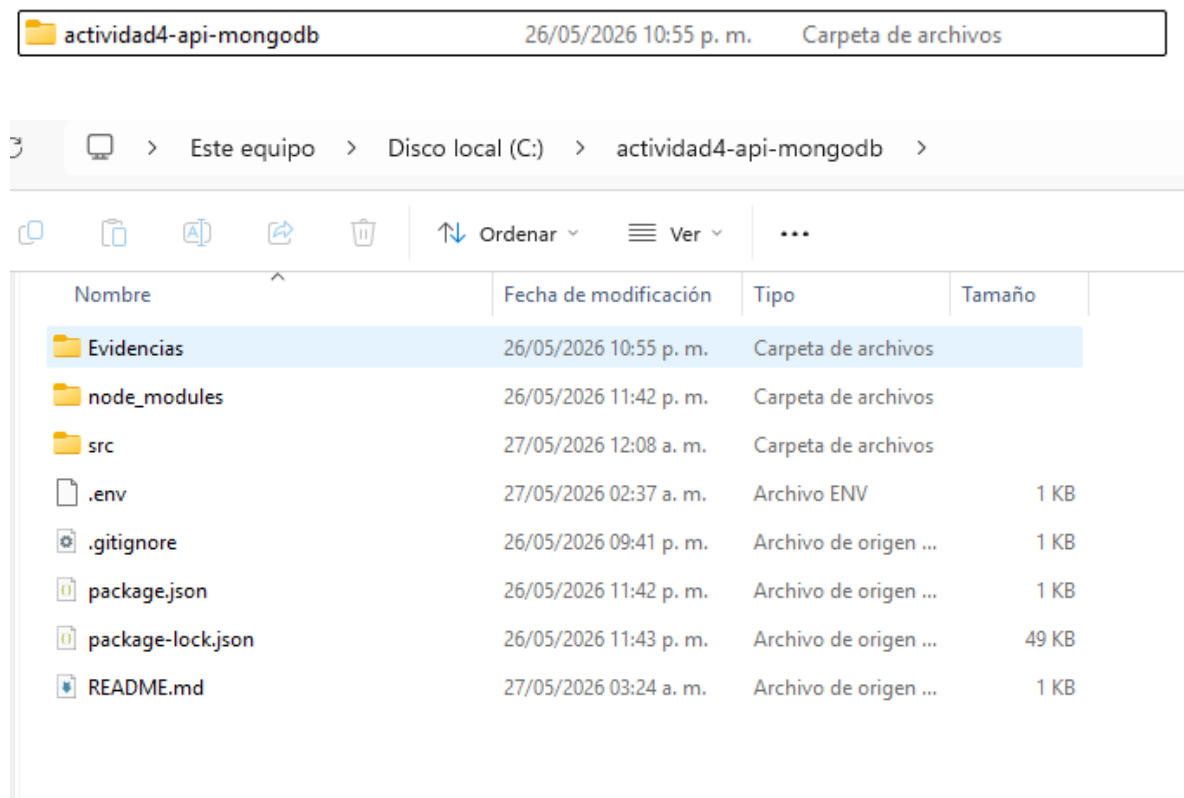
FECHA:

27/05/2026

Introducción

En esta actividad se desarrolló una API RESTful utilizando Node.js, Express y MongoDB con Mongoose para implementar operaciones CRUD persistentes. El objetivo principal fue conectar una aplicación back-end con una base de datos real, permitiendo almacenar, consultar, actualizar y eliminar información de manera permanente. Además, se trabajó con una estructura organizada por responsabilidades, utilizando controladores, rutas, modelos, middlewares y variables de entorno para mejorar el mantenimiento y la organización del proyecto. Finalmente, se realizaron pruebas mediante Postman y MongoDB Compass para comprobar el correcto funcionamiento de todos los endpoints y la persistencia de los datos.

Lo primero que realice fue crear una copia del proyecto actividad3-api-crud para luego cambiarle el nombre a actividad4-api-mongodb y no romper nada del proyecto anterior y seguir trabajando.



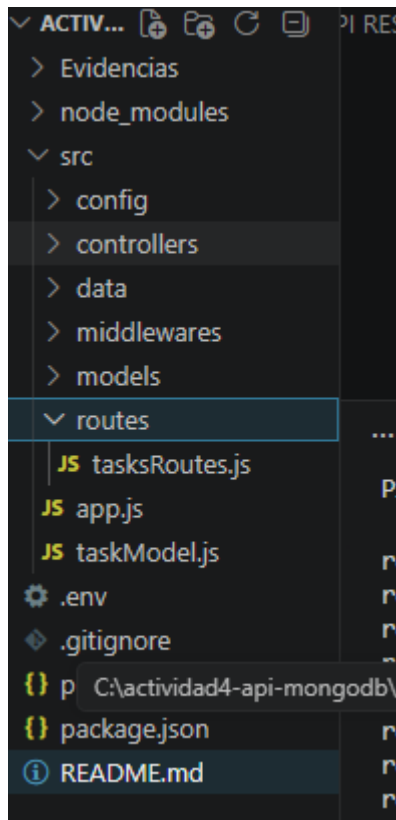
Instalar Mongoose y dotenv en la terminal

```
PS C:\actividad4-api-mongodb> npm install mongoose dotenv
added 2 packages, and audited 95 packages in 4s
30 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
```

Crear estructura nueva

Se agrego:

src/config
src/models

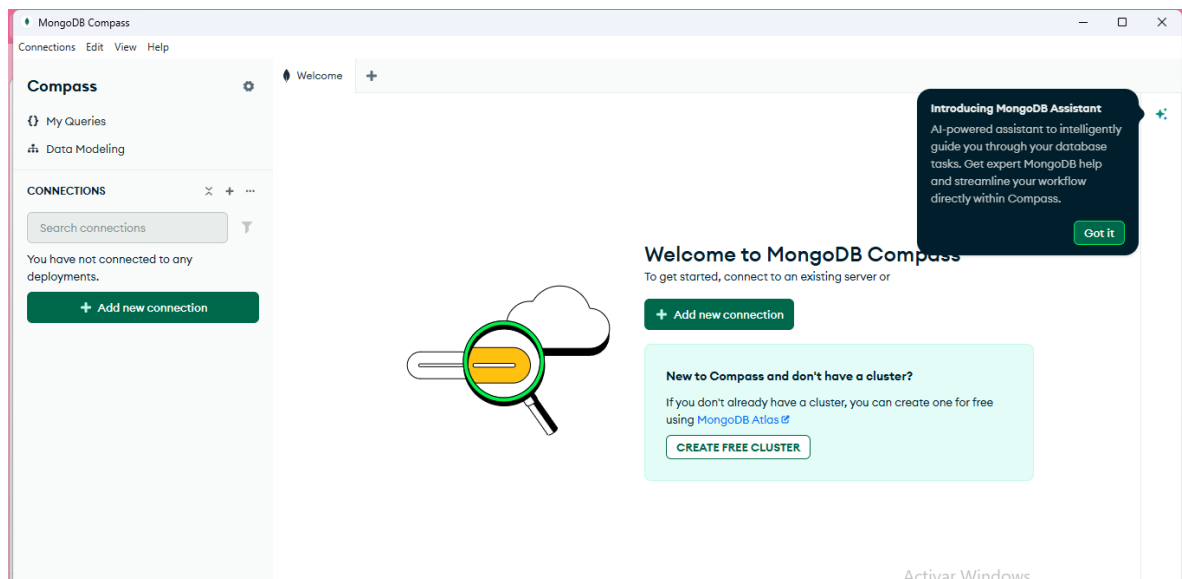
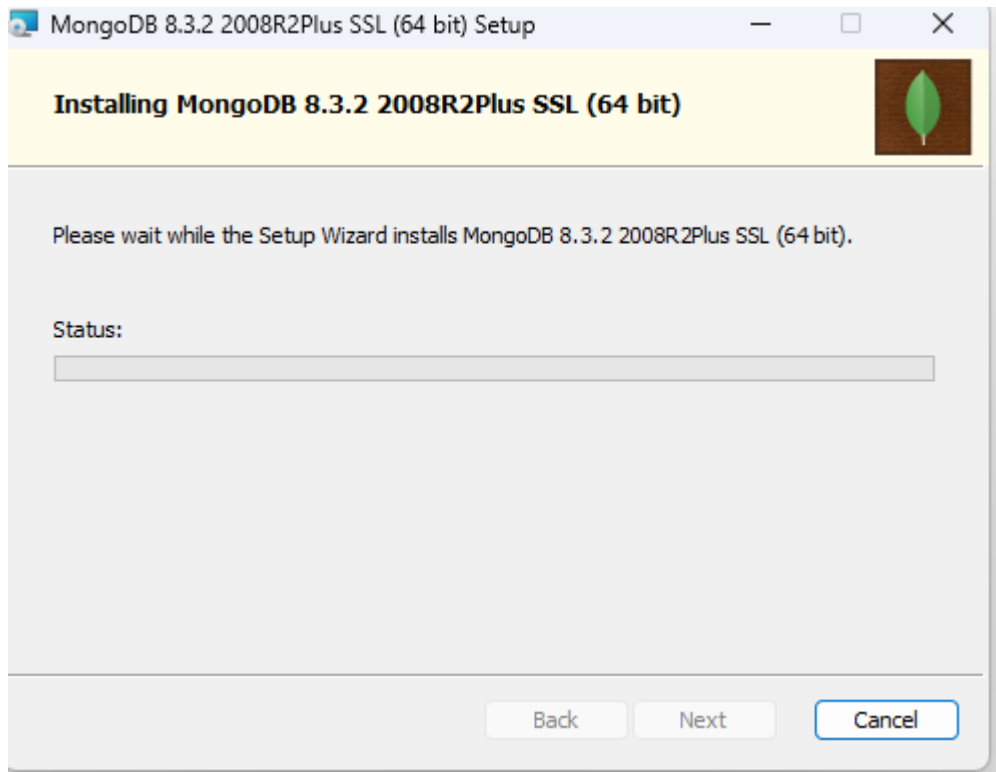


Crear archivo .env



Instalación de MongoDB Community Server

Se instaló MongoDB Community Server para utilizar una base de datos local y permitir la persistencia real de los datos mediante MongoDB y Mongoose.



Se configuró MongoDB Compass para conectarse al servidor local MongoDB utilizando la dirección `mongodb://127.0.0.1:27017`.

New Connection

Manage your connection settings

URI ⓘ Edit Connection String ☒

Name

Color

☐ Favorite this connection
Favoriting a connection will pin it to the top of your list of connections

> Advanced Connection Options

How do I find my connection string in Atlas?

If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.

[See example](#)

How do I format my connection string?

[See example](#)

Cancel

Save

Connect

Save & Connect

Copiar la cadena de conexión

Se estableció correctamente la conexión entre Express y MongoDB mediante Mongoose utilizando variables de entorno en el archivo .env.

```
.env
1 PORT=3000
2 MONGODB_URI=mongodb://127.0.0.1:27017/tasksdb

PS C:\actividad4-api-mongodb> npm run dev

> actividad3-api-crud@1.0.0 dev
> nodemon src/app.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/app.js`
◇ injected env (2) from .env // tip:  override existing { override: true }
Servidor corriendo en puerto 3000
MongoDB conectado
```

Crear Archivo:

src/config/db.js

El archivo db.js se encarga de establecer la conexión entre la aplicación Express y la base de datos MongoDB utilizando Mongoose. Además, permite centralizar la configuración de la conexión para facilitar la organización y mantenimiento del proyecto.

```
src > config > JS db.js > ...
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4
5      try {
6
7          await mongoose.connect(process.env.MONGODB_URI);
8
9          console.log('MongoDB conectado');
10
11      } catch (error) {
12
13          console.log(error);
14
15          process.exit(1);
16
17      }
18
19  };
20
21  module.exports = connectDB;
```

Crear archivo

src/models/taskModel.js

Se creó el modelo Task utilizando Mongoose para definir la estructura del recurso y sus validaciones.

```
src > models > JS taskModel.js > ...
1  const mongoose = require('mongoose');
2
3  const taskSchema = new mongoose.Schema({
4
5      title: {
6          type: String,
7          required: true,
8          minlength: 3
9      },
10
11     description: {
12         type: String,
13         required: true
14     },
15
16     done: {
17         type: Boolean,
18         default: false
19     },
20
21     priority: {
22         type: String,
23         required: true
24     },
25 }
```



```

26     category: {
27       type: String,
28       required: true
29     },
30
31     dueDate: {
32       type: Date,
33       required: true
34     }
35   }, {
36     timestamps: true
37   });
38
39 module.exports = mongoose.model('Task', taskSchema);

```

```

src > JS taskModel.js > taskSchema > done
1  const mongoose = require('mongoose');
2
3  const taskSchema = new mongoose.Schema({
4
5    title: {
6      type: String,
7      required: true,
8      minlength: 3
9    },
10
11    description: {
12      type: String,
13      required: true
14    },
15
16    done: {
17      type: Boolean,
18      default: false
19    },
20
21    priority: {
22      type: String,
23      required: true
24    },
25

```

Crear archivo app.js dentro de src

El archivo app.js funciona como el archivo principal de la aplicación. En él se configura Express, los middlewares, las rutas del sistema y la conexión con la base de datos, además de iniciar el servidor en el puerto definido.

```
require('dotenv').config();

const express = require('express');

const connectDB = require('./config/db');

const tasksRoutes = require('./routes/tasksRoutes');
const notFound = require('./middlewares/notFound');
const errorHandler = require('./middlewares/errorHandler');

const app = express();

connectDB();

app.use(express.json());

app.use('/tasks', tasksRoutes);

app.use(notFound);

app.use(errorHandler);

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Servidor corriendo en puerto ${PORT}`);
});
```

En seguida se abrió src/controllers/tasksController.js para reemplazar código de JavaScript porque ahora MongoDB guardara todo.

```
const Task = require('./models/taskModel');

// GET /tasks
exports.getAllTasks = async (req, res) => {

  try {

    const tasks = await Task.find();
```

```
    res.status(200).json({
      ok: true,
      data: tasks
    });

  } catch (error) {

    res.status(500).json({
      ok: false,
      error: error.message
    });

  }

};

// GET /tasks/:id
exports.getTaskById = async (req, res) => {

  try {

    const task = await Task.findById(req.params.id);

    if (!task) {

      return res.status(404).json({
        ok: false,
        error: 'Tarea no encontrada'
      });

    }

    res.status(200).json({
      ok: true,
      data: task
    });

  } catch (error) {

    res.status(500).json({
      ok: false,
      error: error.message
    });

  }

}
```

```

};

// POST /tasks
exports.createTask = async (req, res) => {

  try {

    const newTask = await Task.create(req.body);

    res.status(201).json({
      ok: true,
      data: newTask
    });

  } catch (error) {

    res.status(400).json({
      ok: false,
      error: error.message
    });

  }

};

// PUT /tasks/:id
exports.updateTask = async (req, res) => {

  try {

    const updatedTask = await Task.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );

    if (!updatedTask) {

      return res.status(404).json({
        ok: false,
        error: 'Tarea no encontrada'
      });

    }

  }

};

```

```
    res.status(200).json({
      ok: true,
      data: updatedTask
    });

  } catch (error) {

    res.status(400).json({
      ok: false,
      error: error.message
    });

  }

};

// DELETE /tasks/:id
exports.deleteTask = async (req, res) => {

  try {

    const deletedTask = await Task.findByIdAndDelete(req.params.id);

    if (!deletedTask) {

      return res.status(404).json({
        ok: false,
        error: 'Tarea no encontrada'
      });

    }

    res.status(204).send();

  } catch (error) {

    res.status(500).json({
      ok: false,
      error: error.message
    });

  }

};
```

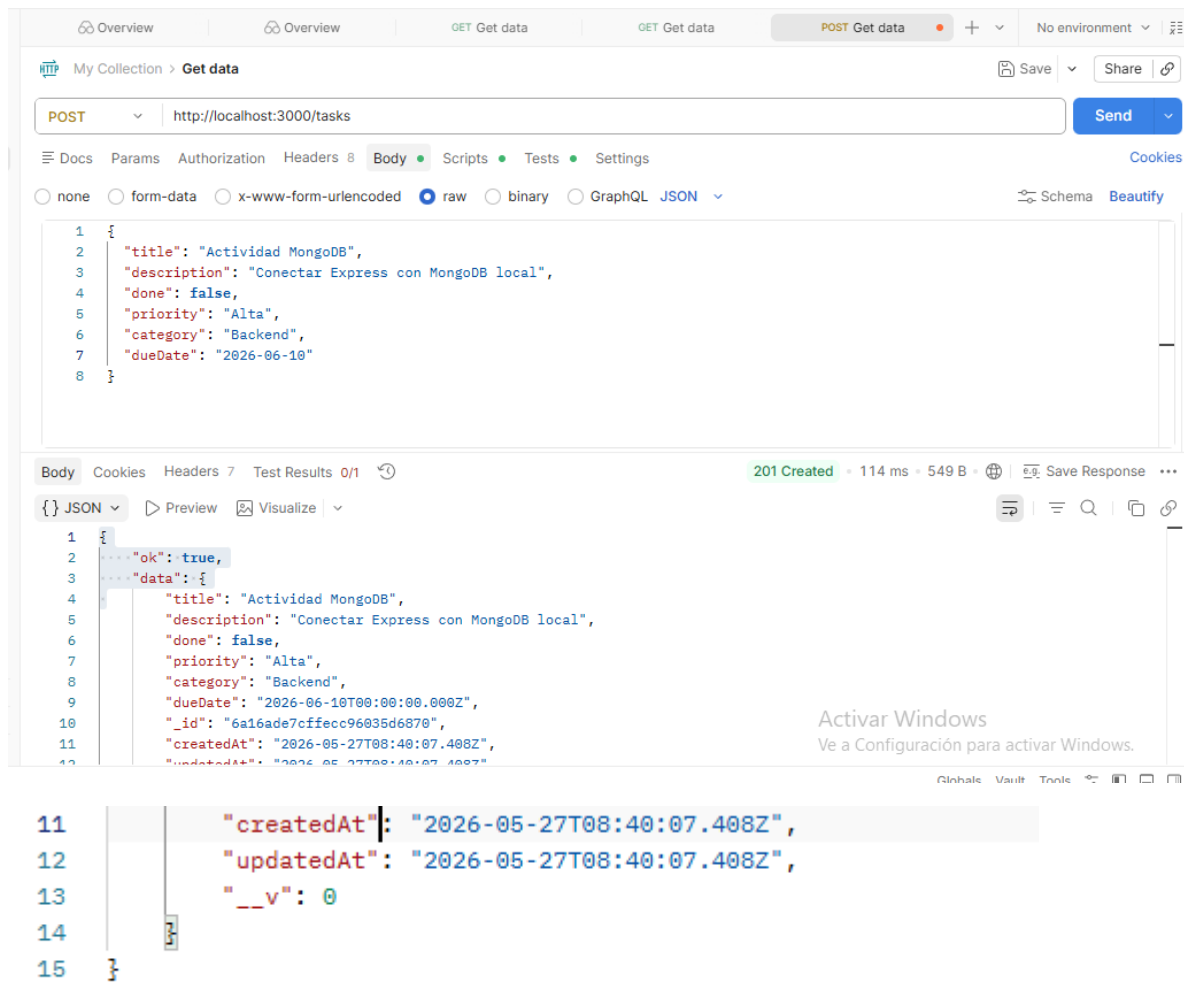
En seguida se mostraran las pruebas en Postman.

POST — Crear tarea

Método:

POST

Se realizó una prueba del endpoint POST /tasks para registrar una nueva tarea en la base de datos MongoDB. La petición se envió mediante Postman utilizando formato JSON. Como resultado, el sistema creó correctamente el registro y devolvió una respuesta exitosa junto con el identificador único (_id) generado automáticamente por MongoDB.



GET ALL

Método:

GET

Se probó el endpoint GET /tasks para consultar todas las tareas almacenadas en la base de datos. La respuesta mostró correctamente la información guardada en MongoDB, confirmando que la persistencia de datos funciona correctamente y que el sistema puede

recuperar

los

registros

almacenados.

My Collection > Get data

GET http://localhost:3000/tasks

Docs Params Authorization Headers 6 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1

Body Cookies Headers 7 Test Results 1/1

200 OK • 52 ms • 546 B

JSON Preview Visualize

```
1 {
2   "ok": true,
3   "data": [
4     {
5       "_id": "6a16ade7cffecc96035d6870",
6       "title": "Actividad MongoDB",
7       "description": "Conectar Express con MongoDB local",
8       "done": false,
9       "priority": "Alta",
10      "category": "Backend",
11      "dueDate": "2026-06-10T00:00:00.000Z",
12      "createdAt": "2026-05-27T08:40:07.408Z",
13      "updatedAt": "2026-05-27T08:40:07.408Z",
14      "__v": 0
15    }
16  ]
17 }
```

PUT — Actualizar tarea

Método:

PUT

Se realizó una prueba del endpoint PUT /tasks/:id para actualizar la información de una tarea existente. Se modificaron algunos campos del registro y el sistema devolvió la información actualizada correctamente, comprobando que la edición de datos en

MongoDB funciona de manera adecuada.

The screenshot displays a REST client interface with a PUT request to `http://localhost:3000/tasks/6a16ade7cffecc96035d6870`. The request body is a JSON object representing a task. The response is a 200 OK status with a JSON body indicating successful creation of the task.

Request:

```
1 {
2   "title": "Actividad MongoDB Actualizada",
3   "description": "CRUD conectado a MongoDB",
4   "done": true,
5   "priority": "Media",
6   "category": "Backend",
7   "dueDate": "2026-06-15"
8 }
```

Response:

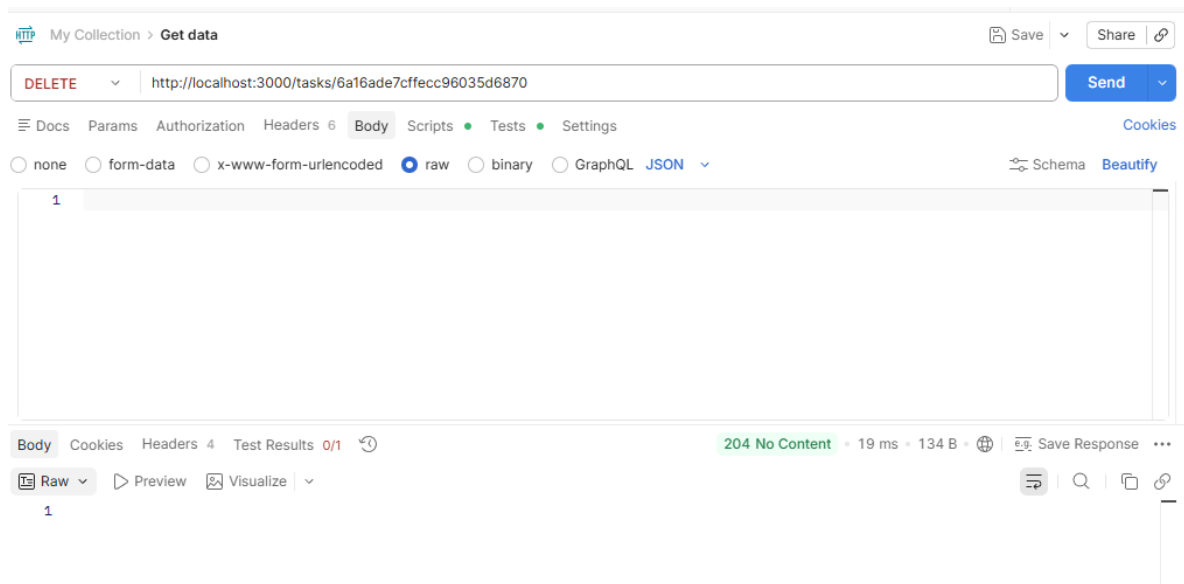
```
1 {
2   "ok": true,
3   "data": {
4     "_id": "6a16ade7cffecc96035d6870",
5     "title": "Actividad MongoDB Actualizada",
6     "description": "CRUD conectado a MongoDB",
7     "done": true,
8     "priority": "Media",
9     "category": "Backend",
10    "dueDate": "2026-06-15T00:00:00.000Z",
11    "createdAt": "2026-05-27T08:40:07.408Z",
12    "updatedAt": "2026-05-27T08:47:11.936Z",
13    "__v": 0
14  }
15 }
```

DELETE — Eliminar tarea

Método:

DELETE

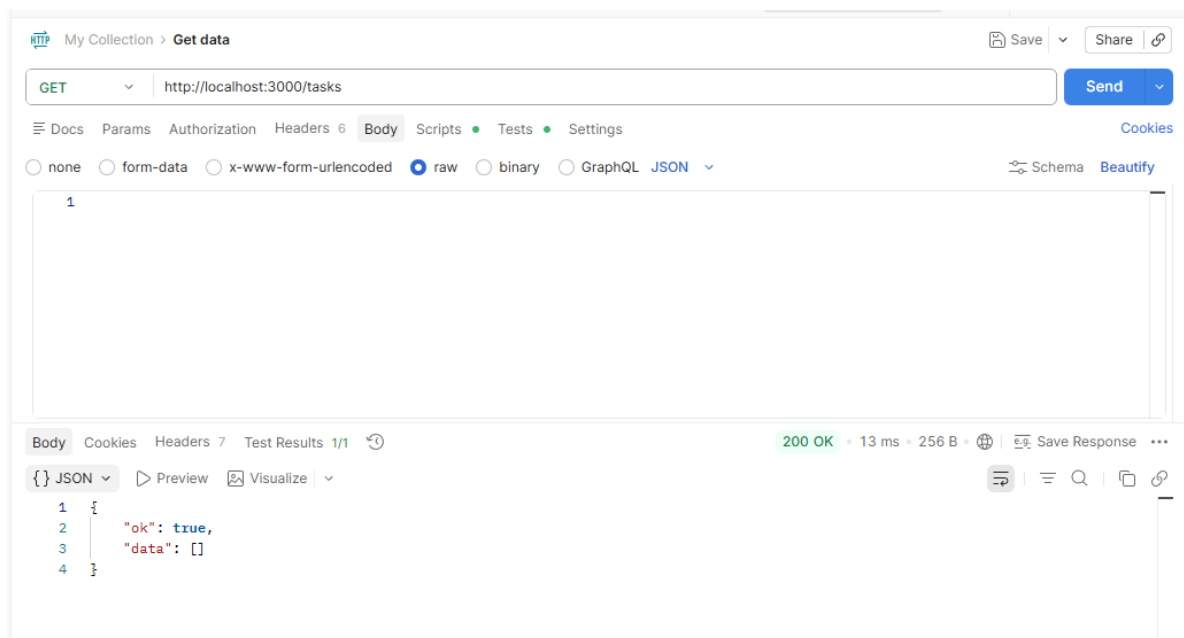
Se ejecutó el endpoint DELETE `/tasks/:id` para eliminar una tarea almacenada en la base de datos. El sistema respondió correctamente indicando que la eliminación fue exitosa, validando así el correcto funcionamiento de la operación DELETE en MongoDB.



Posteriormente hacer otro GET <http://localhost:3000/tasks>

Eso demuestra:

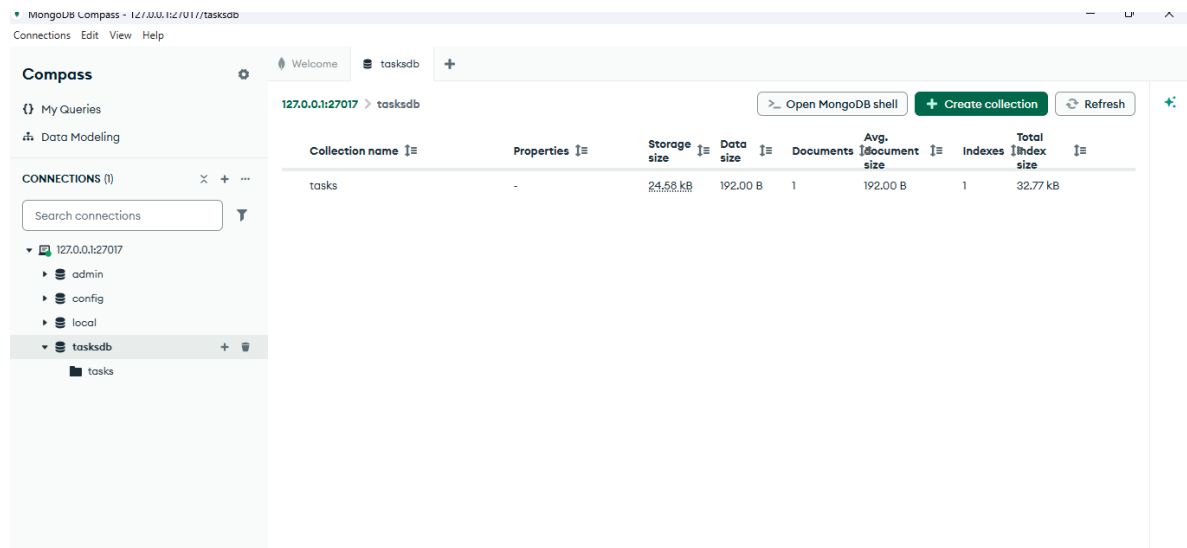
- Eliminación
- persistencia real
- CRUD completo



MongoDB Compass

Evidencia de persistencia

Se verificó en MongoDB Compass que los registros creados desde la API RESTful fueron almacenados correctamente dentro de la base de datos tasksdb y la colección tasks. Esto confirma que la aplicación trabaja con persistencia real utilizando MongoDB y Mongoose.



Actualizar GitHub

```
PS C:\actividad4-api-mongodb> git add .
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it
PS C:\actividad4-api-mongodb> git commit -m "Actividad 4 MongoDB"
[main 4a1963f] Actividad 4 MongoDB
 8 files changed, 408 insertions(+), 94 deletions(-)
 create mode 100644 .env
 create mode 100644 src/config/db.js
 create mode 100644 src/models/taskModel.js
 create mode 100644 src/taskModel.js
PS C:\actividad4-api-mongodb> git push
Enumerating objects: 21, done.
Counting objects: 100% (21/21), done.
Delta compression using up to 12 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (14/14), 4.61 KiB | 277.00 KiB/s, done.
Total 14 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/yoselinolis0104-maker/actividad3-api-crud.git
 b5a2857..4a1963f main -> main
PS C:\actividad4-api-mongodb>
```

Conclusión

Con el desarrollo de esta actividad fue posible comprender el funcionamiento de una API RESTful conectada a una base de datos MongoDB mediante Mongoose. Se logró implementar correctamente las operaciones CRUD persistentes utilizando Express y MongoDB local, además de validar la conexión y el almacenamiento real de la información. También se reforzó el uso de buenas prácticas como la organización del proyecto por carpetas, el uso de variables de entorno y el manejo de respuestas HTTP. Las pruebas realizadas en Postman y MongoDB Compass permitieron verificar que el sistema funciona correctamente y que los datos permanecen almacenados en la base de datos.